DRRC Mid-Term Viva Notes

Oct 6, 2026 · @Yash

1. The project in one minute

DRRC (Disaster Response Resource Coordinator) is a web app for a disaster control room. It takes in emergency reports, uses AI agents to sort and rank them, and suggests which unit to send. A human commander always makes the final decision.

Opening line for the viva (learn this):

> "In a flood or any disaster, the control room is flooded with reports: phone calls, social media posts, many about the same place. Someone has to sort them, decide what is most urgent, and pick which team to send. Our project does that work. An AI agent reads each report and classifies it, reports about the same event are merged using map queries, every incident gets a priority score with a reason, and the system suggests the best unit using real road travel time. But the commander always makes the final click, and every decision is recorded."

The problem it solves

- **Too many reports, no order.** We give each incident a 0–100 priority score with a written reason.
- **The same event is reported many times.** Reports of the same type within 500 m and 60 minutes merge into one incident.
- **Nearest unit is not always fastest.** We use real road ETAs from OSRM, not straight-line distance.
- **AI can be wrong.** A human approves every dispatch, low-confidence results are marked "Needs checking", and every step is written to an audit log.

Key facts to remember

Fact	Value
Demo city	Chennai, monsoon flood scenario
Roles	Commander (decides), Admin (manages units, read-only dashboard), Responder (field unit)
Agents	Triage Agent, Dispatch Agent, Routing Agent (Supervisor Agent planned for end-term)

Fact	Value
Seed data	33 resources, 6 users, 26 mock social media posts
Code size	about 6,500 lines of TypeScript, SQL and CSS
Team	3 students, B.Tech 7th semester minor project

Three words to keep repeating: explainable, human-in-the-loop, geospatial. Every agent gives a plain-language reason, nothing dispatches without a human click, and PostGIS does the location work inside the database.

2. Tech stack and why we chose each piece

Everything is written in **TypeScript**: one language for the browser, the server and the agents. The examiner will often ask "why X and not Y", so learn the right-hand column.

Layer	What we used	Why (say this)
Frontend	**React 19** + TypeScript, built with **Vite 8**	React builds the screen from small reusable components; Vite gives an instant dev server.
Page navigation	**React Router 7**	Different URLs (`/dashboard` , `/reports/new` , `/resources`) show different pages without reloading.
Map	**Leaflet 1.9** + react-leaflet 5, **OpenStreetMap** tiles	Free and open, no API key, works well with GPS coordinates.
Backend	**Node.js 24** + **Express 5**	Same language as the frontend; Express is the most common, simplest web server for Node. Express 5 forwards errors from `async` functions automatically.
Running TypeScript	**tsx**	Runs `.ts` files directly, so the server has no build step.
Agents	**LangGraph.js** (`@langchain/langgraph`)	Lets us draw an agent as a graph of steps with a shared state and branches (AI → fallback → check). Easy to extend into a Supervisor Agent.

Layer	What we used	Why (say this)
AI model	**Google Gemini** (`gemini-3.5-flash`) via `@langchain/google-genai`	Fast, cheap, has a free tier, supports **structured output** (answers in a fixed JSON shape).
Database	**PostgreSQL 17/18 + PostGIS 3**	Reliable relational database; PostGIS adds maps maths (distance, "within 500 m", "nearest first") with spatial indexes.
DB driver	**pg** (node-postgres), plain SQL	No ORM, so every query is visible and explainable; parameters (`$1`) stop SQL injection.
Live updates	**Socket.IO 4**	Server pushes changes to every open screen within about a second; auto-reconnects.
Road routing	**OSRM** (public server `router.project-osrm.org`)	Free routing engine on OpenStreetMap roads; gives real road ETAs and the route line.
Address search	**Nominatim** (OpenStreetMap)	Free geocoding: turns "T. Nagar" into coordinates and back. Called through our server, limited to 1 request per 1.1 s, results cached.
Login	**JWT** + **bcryptjs**	JWT = signed login token, no server sessions needed; bcrypt stores passwords as slow, salted hashes.
Validation	**zod 4**	One schema checks the `.env` file, every request body and even the AI's answer.
Security headers	helmet, cors	Standard safe HTTP headers; only our frontend's address may call the API.
Lint	oxlint	Fast code-quality checker for the client.

How the parts run during development: Vite serves the React app on port **5173** and forwards `/api` and `/socket.io` to the Express server on port **4000** (`client/vite.config.ts`). So the browser talks to one address only.

Configuration lives in `server/.env` and is checked by zod when the server starts (`server/src/config/env.ts`). A missing or wrong value stops the server with a clear message. Key settings: `DATABASE_URL`, `JWT_SECRET` (min 16 chars), `JWT_EXPIRES_IN=12h`, `GEMINI_API_KEY`, `GEMINI_MODEL`, `OSRM_URL`, `NOMINATIM_URL`, `DEDUP_RADIUS_METERS=500`, `DEDUP_WINDOW_MINUTES=60`, `TRIAGE_REVIEW_THRESHOLD=0.6`, `SOCIAL_FEED_INTERVAL_SECONDS=5`.

Likely "why not" questions

- *Why not MongoDB?* Our data is related (reports → incidents → decisions → units) and needs transactions and geospatial indexes. PostgreSQL + PostGIS does both.
- *Why not an ORM (Prisma, Sequelize)?* Plain SQL shows exactly what runs, and PostGIS functions are easier to write directly.
- *Why not Google Maps?* It needs a paid key. OpenStreetMap, OSRM and Nominatim are free and open.
- *Why not polling instead of WebSockets?* Polling wastes requests and is slower; with sockets the server pushes only when something changes.

3. Architecture and the journey of one report

The system has three tiers: a **React frontend** in the browser, an **Express backend** that also runs the AI agents, and a **PostgreSQL + PostGIS database**. The frontend talks to the backend in two ways: **REST** (HTTP + JSON) to load data and do actions, and **WebSockets** (Socket.IO) so the server can **push** changes to every open screen.

Backend layering (same in every feature): `<feature>.routes.ts` checks the role and validates the input → `<feature>.service.ts` applies the rules, runs SQL, writes the audit log and broadcasts the change → database. The agents live in `server/src/agents/` and are called by the services.

The full journey (learn this order; it answers most "how does it work" questions)

1. **A report arrives.** Either a commander/admin fills the form (`POST /api/reports`) or the mock social feed posts one every 5 seconds. Both call the same function: `ingestReport()` in `server/src/modules/reports/reports.service.ts`.
2. **Save first.** The raw report is inserted into `reports` straight away, so nothing is lost even if a later step fails.

3. **Triage Agent.** `runTriage(text)` runs the LangGraph graph: try Gemini → if it fails, keyword rules → check confidence. Output: type, severity 1–5, confidence 0–1, short title, reason, and `needsReview`. Written to the audit log.
4. **Duplicate check (one at a time).** SQL looks for an active incident of the **same type**, **within 500 m** (`ST_DWithin`) and with a report in the **last 60 minutes**, nearest first (`<->`).
   - Found → **merge**: report count +1, keep the highest severity, recompute urgency.
   - Not found → **create** a new incident with an urgency score.
5. **Push.** `publishIncident()` sends `incident:updated` to every commander and admin screen. The incident appears in the queue and on the map.
6. **Commander reviews** (if marked "Needs checking"): confirms or corrects type and severity (`PUT /api/incidents/:id/classification`).
7. **Commander clicks "Find the best unit".** `POST /api/dispatch/incidents/:id/recommend` → Dispatch Agent: PostGIS nearest free units of the right type → capability and load check → Routing Agent gets road ETAs from OSRM → score and rank → plain-language reason. Saved as a **pending** row in `dispatch_decisions`. Nothing is sent yet.
8. **Commander decides.** "Send" = accept (`POST /api/dispatch/decisions/:id/accept`) or "Choose another unit" = override with a reason (`/override`). In **one transaction with row locks**: unit → `en_route` , incident → `dispatched` , decision → `accepted` / `overridden` . Then audit log, broadcast, and `task:assigned` to the responder of that unit.
9. **Unit arrives.** "Mark as arrived" → incident and unit become `on_scene` ; the unit's position moves to the incident.
10. **Done.** "Mark as resolved" → incident `resolved` , unit back to `available` . A false alarm can be closed without sending any unit.

Status words you will be asked about

Thing	Statuses (database value → what the UI says)
Incident	`open` (Waiting) → `dispatched` (Unit on the way) → `on_scene` (On scene) → `resolved` (Resolved)
Resource (unit)	`available` → `en_route` → `on_scene` → back to `available` ; or `out_of_service`
Dispatch decision	`pending` → `accepted` or `overridden` ; an older pending one becomes `superseded` when a new recommendation is made

Three ideas that run through everything: agents only **recommend** (human in the loop); every agent decision and human action goes into `audit_log`; every change is **pushed** live over Socket.IO, so no screen ever polls.

4. Database: tables, PostGIS and key SQL

All data lives in **6 tables** in PostgreSQL, defined in `server/src/db/migrations/001_init.sql`. PostGIS is switched on with `CREATE EXTENSION IF NOT EXISTS postgis;`.

Table	What one row is	Important columns
`reports`	One raw message (form, social post, weather alert later)	`source`, `text`, `location`, `address`, `image_url` (for end-term), `reporter`, `reported_at`, `incident_id` (which incident it was merged into)
`incidents`	One real-world event, built from 1+ reports	`title`, `type`, `severity` 1–5, `urgency_score` 0–100, `urgency_reason`, `status`, `location`, `report_count`, `confidence` 0–1, `needs_review`, `classified_by` (ai / rules / commander), `triage_reason`, `created_at`, `last_report_at`, `dispatched_at`, `resolved_at`
`resources`	One unit: ambulance, team, truck or shelter	`code` (AMB-04), `type`, `status`, `location`, `home_base`, `capacity`, `current_load`, `capabilities TEXT[]`, `assigned_incident_id`, `retired_at`
`dispatch_decisions`	One recommendation by the Dispatch Agent and the human's answer	`recommended_resource_id`, `resource_id` (unit actually sent), `agent_recommendation JSONB` (all ranked candidates), `rationale`, `human_action`, `override_reason`, `decided_by`, `decided_at`
`users`	One login	`name`, `email` (unique), `password_hash`, `role`, `resource_id` (only for responders: their unit)

Table	What one row is	Important columns
`audit_log`	One logged event	`actor_type` (agent / human / system), `actor_name`, `action` (e.g. `dispatch.accepted`), `entity_type` + `entity_id`, `summary` (one readable sentence), `details JSONB`, `created_at`

Relationships: a report belongs to one incident; an incident has many reports and many dispatch decisions; a decision points to the recommended unit and the unit actually sent; a resource can point to the incident it is assigned to; a responder user points to their unit.

PostGIS in plain words

- **`GEOGRAPHY(POINT, 4326)`**: every location column. 4326 is the standard GPS system (latitude/longitude). "Geography" means distances are measured in **metres on the curved earth**, not in degrees.
- **Writing a point:** `ST_MakePoint(lng, lat)::geography`. **Longitude comes first** (x, then y). A classic bug if swapped.
- **Reading a point:** `ST_Y(location::geometry) AS lat`, `ST_X(location::geometry) AS lng`.
- **`ST_DWithin(a, b, 500)`**: true if a and b are within 500 m. Used for **duplicate detection**.
- **`ST_Distance(a, b)`**: distance in metres. Shown in audit messages and dispatch reasons.
- **`ORDER BY a <-> b`**: the **KNN (k-nearest-neighbour) operator**: sort by distance, nearest first. Used to find the nearest units.
- **GIST index** on every `location` column (e.g. `CREATE INDEX resources_location_gix ON resources USING GIST (location)`). It works like a map divided into boxes, so the database checks only nearby boxes instead of every row. This is what the SRS means by "proper geospatial indexing, not naive distance loops".

Data rules enforced by the database itself

- **`CHECK` constraints** on every status, type and role (e.g. `severity BETWEEN 1 AND 5`, `current_load <= capacity`). The same lists live in `server/src/domain.ts`, so code and DB agree.
- **Foreign keys** with `ON DELETE SET NULL` / `CASCADE`.
- **Unique** `resources.code` and `users.email`.

Migrations and seed data

- `npm run db:migrate` runs every `.sql` file in `migrations/` that has not run yet, in name order, each inside a transaction, and records it in a `schema_migrations` table (`server/src/db/migrate.ts`). A schema change = a new numbered file, never editing `001_init.sql`.
- `npm run db:reset` drops everything, migrates and seeds. Do this before every demo.
- **Seed** (`server/src/db/seed.ts`): **33 resources** at real Chennai localities: 9 ambulances (AMB-05 is out of service), 4 fire teams, 4 water rescue teams (BOAT), 2 search and rescue teams, 2 road crews, 2 electricity crews, 4 supply trucks, 6 relief camps (shelters). Plus **6 users** (1 commander, 1 admin, 4 responders linked to AMB-01, FIRE-01, BOAT-01, TRK-01). Passwords are hashed with bcrypt (cost 10).

Connecting from code (`server/src/db/pool.ts`)

- **One shared connection pool** (`pg.Pool`) for the whole app; opening a new connection per request would be slow.
- `query(sql, params)` for single statements; `withTransaction(work)` runs `BEGIN` → your queries → `COMMIT`, or **`ROLLBACK`** if anything throws (all-or-nothing).
- **Parameterised queries** (`$1, $2`) everywhere: values are sent separately from the SQL text, so SQL injection is impossible.
- NUMERIC columns are parsed into JS numbers (pg returns them as strings by default).
- On startup `index.ts` runs `SELECT 1`; if the database is unreachable the server exits with "Cannot connect to the database. Check DATABASE_URL".

5. Backend basics: how the server is built

The backend is one Node.js process that serves the REST API, the WebSocket connection and the agents, all on **port 4000**.

Startup (`server/src/index.ts`)

1. Load and validate `.env` with zod (`config/env.ts`); stop with a clear error if anything is wrong.
2. Test the database with `SELECT 1`; stop if it fails.
3. `createApp()` builds the Express app; `http.createServer(app)` wraps it.
4. `initRealtime(server)` attaches Socket.IO to the **same** HTTP server (same port).
5. `server.listen(4000)` and log whether triage runs in **AI mode** or **keyword mode** (no Gemini key).
6. On Ctrl+C (`SIGINT` / `SIGTERM`), close the server and the DB pool cleanly.

The app and its routes (`server/src/app.ts`)

Middleware runs in order for every request:

1. `helmet()`: safe HTTP security headers.
2. `cors({ origin: CLIENT_ORIGIN })`: only our frontend may call the API from a browser.
3. `express.json({ limit: '1mb' })`: reads JSON bodies.
4. `morgan('dev')`: one log line per request.
5. **Public:** `GET /api/health`, `/api/auth` (login).
6. **`app.use('/api', requireAuth)`**: everything after this line needs a valid token.
7. Feature routers: `/api/system` (AI or rules mode), `/api/reports`, `/api/incidents`, `/api/resources`, `/api/dispatch`, `/api/feeds`, `/api/geocode`.
8. `notFoundHandler` (404) and `errorHandler` (turns errors into JSON) at the end.

Middleware = a function that runs before the route handler and can stop the request or add data to it (like `req.user`). Order matters.

Main API endpoints

Method + URL	Who	Does
`POST /api/auth/login`	anyone	email + password → `{ token, user }`
`GET /api/auth/me`	logged in	restores the session after a page reload
`POST /api/reports`	commander, admin	log a manual report → runs the full pipeline
`GET /api/incidents` (`?view=resolved`)	commander, admin	active queue (or recently resolved)
`GET /api/incidents/:id`	commander, admin	incident + its reports + history
`PUT /api/incidents/:id/classification`	commander	confirm/correct type and severity
`POST /api/incidents/:id/status`	commander	`on_scene` or `resolved`
`POST /api/dispatch/incidents/:id/recommend`	commander	run the Dispatch Agent

Method + URL	Who	Does
`POST /api/dispatch/decisions/:id/accept`	commander	send the recommended unit
`POST /api/dispatch/decisions/:id/override`	commander	send another unit, with optional reason
`GET /api/dispatch/route`	commander, admin	road route line for the map
`/api/resources` (GET/POST/PUT, retire/restore)	admin edits; others read	resource inventory
`/api/feeds/social/...`	commander, admin	start / pause / reset the mock social feed
`/api/geocode/search`, `/reverse`	logged in	address ↔ coordinates via Nominatim

Login with JWT (`modules/auth/auth.routes.ts`, `middleware/auth.ts`)

1. User sends email + password. zod checks the format and lower-cases the email.
2. Server finds the user and checks the password with **`bcrypt.compare(password, hash)`**.
3. Wrong email **or** wrong password gives the **same** message "Incorrect email or password." so attackers cannot discover which emails exist.
4. Server creates a **JWT** with `jwt.sign({ id, name, role, resourceId }, JWT_SECRET, { expiresIn: '12h' })`.
5. The browser stores the token and sends it on every request as `Authorization: Bearer <token>`.
6. `requireAuth` verifies the signature and expiry with `jwt.verify` and puts the user on `req.user`. Invalid → 401 "Your session has expired. Please log in again."
7. `requireRole('commander')` checks `req.user.role`; wrong role → **403** "You do not have permission to do this."

What is a JWT? A signed text token with three parts: header, payload (our user info), signature. The server can trust it without storing sessions, because only the server knows the secret that made the signature. It is **signed, not encrypted**, so we never put passwords in it.

What is bcrypt? A slow hashing function with a random salt. Even if the database leaks, the original passwords cannot be read back, and guessing is slow.

Validation (`middleware/validate.ts`)

`validate(schema, data)` checks data with a **zod** schema and returns clean, typed data, or throws a **400** with the first problem, e.g. "text: Please describe what happened (at least 10 characters)". Used for every body, URL param and query string. Example: manual report text must be 10–2000 characters, lat between -90 and 90.

Errors (`utils/httpError.ts`, `middleware/errorHandler.ts`)

- Services just **throw** `badRequest` (400), `unauthorized` (401), `forbidden` (403), `notFound` (404) or `conflict` (409) with a user-friendly message.
- **Express 5** forwards errors from `async` handlers automatically, so routes have **no try/catch**.
- `errorHandler` sends `{ error: message }` with the right status. Unknown errors are logged and the user gets a safe generic 500 message (no stack traces leak).
- **409 Conflict** is used for "someone else already did this", e.g. "AMB-04 is no longer available. Ask the agent for a new recommendation."

6. Triage Agent: classifying each report

The Triage Agent reads one report's text and returns **type, severity (1–5), confidence (0–1), a short title, a reason, the method used (ai / rules) and `needsReview`**. Files: `server/src/agents/triage/`.

The LangGraph graph (`triageGraph.ts`)

LangGraph builds an agent as a **graph**: **nodes** are functions, **edges** say which node runs next, and all nodes read and update one shared **state** object.

- **State:** `{ text, classification, method, aiError, needsReview }`
- **Node 1 `classify_with_ai`:** if there is no API key, it records `aiError` and returns. Otherwise it calls Gemini; any error is caught and logged.
- **Conditional edge:** if a classification exists → go to `check_confidence`; otherwise → `classify_with_rules`. This is decided **at run time**.
- **Node 2 `classify_with_rules`:** keyword fallback.
- **Node 3 `check_confidence`:** `needsReview = (method is rules) OR (confidence < 0.6)`. If the AI is unsure, a sentence like "Confidence is only 45%, so a person should check it." is added to the reason.
- Flow: `START → classify_with_ai → (classify_with_rules) → check_confidence → END`. Called with `runTriage(text)`, which does `triageGraph.invoke({ text })`.

Why LangGraph and not a simple if/else? Today it could be an if/else. LangGraph makes the steps, state and branches explicit and visible, gives a standard way to add more agents, and the end-term **Supervisor Agent** will join Triage → Dispatch → Routing into one bigger graph.

The AI classifier (`llmClassifier.ts`)

- Uses `ChatGoogleGenerativeAI` from `@langchain/google-genai` , model from `GEMINI_MODEL` (default `gemini-3.5-flash`).
- **`temperature: 0`** : same input gives the same answer (no creativity needed).
- **`maxRetries: 1`** and **15-second timeout**: fail fast and fall back to rules instead of making the commander wait.
- **`withStructuredOutput(zodSchema)`** : forces Gemini to answer in exactly this shape: `type` (one of the 8 types), `severity` (integer 1–5), `confidence` (0–1), `title` (≤ 80 chars), `reason` (≤ 400 chars). The answer is checked again with `classificationSchema.parse()` . So we never parse free text.
- **System prompt** tells it: you are the triage officer of a Chennai control room; here are the 8 types and the severity scale; base the answer only on the text; if several problems, pick the most life-threatening; reports may use Indian English or Tamil words in English letters; use confidence below 0.6 if vague or joking.
- The model object is created once and reused.
- **Important honesty point:** the Gemini API key has not been added yet, so the **AI path has not been run live**. The demo runs in **keyword mode**. `npm run ai:check` tests a key with one sample report.

The keyword fallback (`ruleClassifier.ts`)

Keeps the system working with no internet or no key.

1. **Type:** count keyword hits per type (e.g. flood: "flooded", "waist deep", "water entered"; fire: "smoke", "gas leak"). Most hits wins. Ties go to the more life-threatening type: collapse > fire > medical > flood > supply > road > power. No hits → `other` .
2. **Severity:** base value per type (collapse 5, fire 4, medical 3, flood 3, others 2). **+1** if any escalation word ("trapped", "children", "elderly", "sos"…), **+1 more** if 3 or more. **−1** for a calming word ("minor", "no injuries", "receding"), −1 more for 2+. Kept between 1 and 5.
3. **Confidence:** 0.2 if no keyword; otherwise 0.3 + 0.1 per hit, **capped at 0.5**. So rules results are **always** "Needs checking".
4. **Reason:** lists the exact words that matched, e.g. *Keyword match for "Flooding": "waist deep". Severity 4 (Severe), raised because of: "trapped".*
5. Words are matched as whole words with a regex (`\b...\b`), case-insensitive.

Urgency score (`urgency.ts`)

```
\text{score} = \min\left(100,\; 15 \times \text{severity} + \text{type points} + \min(10,\; 2 \times \text{extra reports})\right)
```

Part	Points
Severity × 15	15 to 75
Type: collapse 15, fire 15, medical 12, flood 10, supply shortage 6, road 5, power 5, other 3	3 to 15
2 per confirming report after the first	0 to 10

Worked example: a severity-4 flood with 3 reports = 60 + 10 + 4 = **74**. Reason shown: "Severity 4 gives 60 pts, incident type gives 10 pts, 2 confirming reports add 4 pts."

Why these weights? Severity dominates (lives at risk matter most); type reflects how fast it gets worse; more reports means more confidence it is real. They are simple on purpose so anyone can check the maths, and can be tuned later with analytics.

Where it is used: the queue is sorted by this score; the reason is shown on the dashboard. It is recomputed when a report merges in or the commander corrects the classification.

7. Duplicate detection: many reports, one incident

A new report is merged into an existing incident when it is **the same type, within 500 m, and that incident got a report in the last 60 minutes** (and is not resolved). Code: `attachToIncident()` in `server/src/modules/reports/reports.service.ts`.

The query

```
SELECT id, report_count AS "reportCount",
       ST_Distance(location, ST_MakePoint($2, $3)::geography) AS distance
  FROM incidents
 WHERE status <> 'resolved'
   AND type = $1
   AND ST_DWithin(location, ST_MakePoint($2, $3)::geography, $4)        --
within 500 m
   AND last_report_at >= $5::timestamptz - make_interval(mins => $6)    --
within 60 min
 ORDER BY location <-> ST_MakePoint($2, $3)::geography                  --
nearest first
 LIMIT 1
```

- `$4` = `DEDUP_RADIUS_METERS` (500), `$6` = `DEDUP_WINDOW_MINUTES` (60); both can be changed in `.env`.
- `ST_DWithin` and `<->` both use the **GIST index**, so it stays fast with many incidents.
- **Type `other` is never merged**: two vague reports nearby are not necessarily the same event.

Merge vs create

- **Merge** (`mergeIntoIncident`, in a transaction, incident row locked with `FOR UPDATE`): `report_count + 1`; **keep the highest severity** seen; recompute urgency; confidence becomes the higher one; stays "Needs checking" **only if the new report is also uncertain**; `last_report_at = now()` (so the 60-minute window slides forward); link the report to the incident.
- **If a commander has already reviewed** the incident (`classified_by = 'commander'`), the agent **never overwrites** their severity, and the incident stays reviewed.
- **Create** (`createIncident`, in a transaction): insert the incident with its urgency score and link the report to it.
- Both write an audit entry, e.g. *"Report #13 describes the same event as incident #4 (95 m away), so it was merged. The incident now has 3 reports."*

Why the "one at a time" queue (`oneAtATime()`)

Triage (the slow AI call) runs **in parallel** for many reports, but the **find-duplicate-or-create** step runs **one at a time** through a promise chain:

```
let dedupQueue: Promise<unknown> = Promise.resolve();
function oneAtATime<T>(task: () => Promise<T>): Promise<T> {
  const run = dedupQueue.then(task, task);
  dedupQueue = run.catch(() => undefined);
  return run;
}
```

The problem it prevents (race condition): two reports of the same flood arrive together. Both search, both find nothing (neither incident exists yet), and both create a new incident: a duplicate. Running the step one after another means the second report sees the incident the first one just created.

- `then(task, task)` runs the next task even if the previous one failed.
- `run.catch(() => undefined)` stops one failure from breaking the chain for all later reports.

- **Limit:** this queue is in the memory of one server process. With several servers you would use a database lock (e.g. `pg_advisory_lock`) instead. Mentioned in the known limitations.

Why "same type" and not just "nearby"?

A fire and a flood on the same street are two different problems needing different units. Grouping by type keeps each incident dispatchable by one kind of unit.

Demo tip: the mock social feed (26 posts in `server/src/data/socialPosts.ts`) contains clusters of posts about the same place, so you can show the report count going up and the priority rising.

8. Dispatch and Routing Agents: choosing the best unit

The Dispatch Agent ranks up to **4 suitable free units** by **real road travel time plus penalties** and explains its choice; the Routing Agent supplies the road times. It runs only when the commander clicks **"Find the best unit"**, and it only **recommends**. Files: `server/src/agents/dispatch/dispatchAgent.ts`, `requirements.ts`, `server/src/agents/routing/routingAgent.ts`.

Step 0: what does this incident need? (`requirements.ts`)

Incident type	Unit type	Ideal equipment
Medical	ambulance	advanced life support **only if severity ≥ 4**, else none
Fire	personnel (team)	fire fighting
Flood	personnel	water rescue (boats)
Building collapse	personnel	search and rescue
Road blocked	personnel	heavy equipment
Power outage	personnel	electrical repair
Food / water shortage	supply truck	food and water
Other	personnel or ambulance	none

Shelters are never dispatched (they are places, not vehicles). Missing equipment is **allowed but penalised**, so a unit is still suggested when no fully equipped one is free, with a warning.

Step 1: nearest free units (PostGIS KNN)

```
SELECT ..., ST_Distance(r.location, ST_MakePoint($1, $2)::geography) AS
"straightLineMeters"
  FROM resources r
 WHERE r.type = ANY($3)            -- right unit type
   AND r.status = 'available'      -- free
   AND r.retired_at IS NULL        -- not retired
   AND r.current_load < r.capacity -- has room
 ORDER BY r.location <-> ST_MakePoint($1, $2)::geography   -- nearest first,
uses GIST index
 LIMIT 8
```

No units found → returns `null`, the audit log says "No suitable unit is available right now. The incident stays in the queue."

Step 2: capability check → shortlist of 4

For each of the 8 units, list matched and missing equipment. Sort by **fewest missing**, then by straight-line distance. Keep the top **4** (to limit calls to OSRM).

Step 3: real road ETAs (Routing Agent)

- **One** OSRM **table** request gives travel time and road distance from all 4 units to the incident at once: `/table/v1/driving/{lng,lat;...}?sources=0;1;2;3&destinations=4&annotations=duration,distance`.
- OSRM wants **`lng,lat`** order. Timeout 8 seconds.
- **Fallback** if OSRM is down or finds no road: straight-line distance (haversine formula) **× 1.4** for road bends, at **25 km/h** city speed. Marked `source: 'estimate'` and the reason says "Road routing was unavailable, so times are rough estimates."
- `getRoute()` uses OSRM's **route** service to get the full road path (GeoJSON) for the blue line on the map. GeoJSON is `[lng, lat]`; Leaflet wants `[lat, lng]`, so it is flipped.

Step 4: score and rank (lower = better)

$$\text{score} = \text{ETA minutes} + 10 \times \text{missing capabilities} + 3 \times \frac{\text{current load}}{\text{capacity}}$$

In words: **one missing piece of equipment counts like being 10 minutes further away; a completely full unit like 3 extra minutes.** Candidates are sorted by score; the first is the recommendation.

Step 5: the plain-language reason (`explain()`)

Built from the numbers, never invented. Possible sentences:

- "BOAT-02 recommended: 9 min by road (6.5 km), 2 min faster than BOAT-03."
- If the winner is slower: "BOAT-03 is 2 min closer but lacks water rescue boat."
- "AMB-07 looks nearer on the map but takes longer by road." (straight-line nearest ≠ road fastest)
- "Carries water rescue boat." or "Warning: it does not carry … X does, but is N min further away."
- "It is the only suitable unit available right now."

Saving it (`createRecommendation()` in `dispatch.service.ts`)

- Refuses if the incident is no longer `open` (409 "A unit has already been dispatched to this incident.").
- In a transaction: mark any older **pending** recommendation for this incident as **`superseded`**, then insert the new one as **`pending`** with all candidates stored as **JSONB** and the rationale.
- Audit log (actor "Dispatch Agent"), broadcast `decision:updated` and the incident.

Why not just send the nearest unit? The SRS says nearest is not always best. In Chennai floods, a unit 2 km away across a river or a closed bridge can take longer than one 4 km away. Road ETAs and equipment matter more than map distance.

Why only 8 then 4? KNN is cheap inside the database; OSRM is an outside service over the internet, so we ask it about only the 4 most promising units in one request.

9. Human decision, incident lifecycle and audit trail

Nothing is dispatched until a commander clicks. The decision runs as **one database transaction with row locks**, so two commanders can never send the same unit twice. Code: `decide()` in `server/src/modules/dispatch/dispatch.service.ts`.

Accept or override: what `decide()` does

Inside `withTransaction` (all-or-nothing):

1. **Lock the decision row** (`SELECT … FOR UPDATE`). If it is no longer `pending` → 409 "This recommendation was already answered or replaced. Please refresh it."
2. Pick the unit: **accept** → the recommended unit; **override** → the commander's chosen unit. Overriding with the same unit → 400 "That is the recommended unit. Use "Send" to accept the recommendation."
3. **Lock the incident row**; it must still be `open` , else 409 "This incident already has a unit assigned."
4. **Lock the unit row**; it must exist, not be a shelter, not be retired and be `available` , else 409 "AMB-04 is no longer available. Ask the agent for a new recommendation."
5. Update three rows: unit → `en_route` + `assigned_incident_id` ; incident → `dispatched` + `dispatched_at` ; decision → `accepted` or `overridden` , with `resource_id` , `override_reason` , `decided_by` , `decided_at` .

After the commit: audit entry ("Accepted the recommendation and sent BOAT-02." or "Overrode the recommendation and sent BOAT-03 instead. Reason: …"), broadcast the decision, unit and incident, and send `task:assigned` to every responder linked to that unit.

What is `SELECT … FOR UPDATE` and why?

It **locks the selected rows until the transaction ends**. A second transaction that tries to lock the same row **waits**. When it continues, it re-reads the row and sees the unit is already `en_route` , so it fails with a clear 409 instead of double-booking.

Race example: Commander A and B both click "Send AMB-04" at the same moment. A locks AMB-04 first and commits. B was waiting; it now reads `en_route` and gets "AMB-04 is no longer available." Without the lock, both could read `available` and both would "send" it.

Why lock before reading? If you read first and lock later, the value you read may already be out of date. Lock, then check, then write.

What is a transaction (ACID)?

A group of SQL statements that succeed or fail together. **Atomic** (all or nothing), **Consistent** (constraints always hold), **Isolated** (others don't see half-done work), **Durable** (saved once committed). If step 5 failed half-way, `ROLLBACK` undoes everything, so we never get "unit en route but incident still open".

Lifecycle after dispatch (`updateIncidentStatus()` in `incidents.service.ts`)

Button	Allowed from	Incident becomes	Unit becomes
Mark as arrived	`dispatched` only	`on_scene`	`on_scene`, and its **location moves to the incident**
Mark as resolved	`dispatched` / `on_scene`	`resolved` + `resolved_at`	`available`, unlinked from the incident
Close without sending a unit (false alarm)	`open`	`resolved`	(none)

- Also in a transaction with the incident row locked first.
- On resolve, any still-pending recommendation becomes `superseded`.
- Audit: "Marked the assigned unit as arrived on scene.", "Marked the incident as resolved. Assigned units are available again.", "Closed the incident without sending a unit (no action needed)."
- Units do **not** move along the road yet; they jump to the incident on arrival. Moving vehicles is an end-term task.

Classification review (`reviewClassification()`)

The commander confirms or corrects type and severity. Sets `confidence = 1`, `needs_review = false`, `classified_by = 'commander'`, recomputes urgency. Audit: "Corrected the classification from Flooding (severity 3) to Building collapse (severity 5)." or "Confirmed the Triage Agent's classification." Later merges never overwrite a commander's classification.

Audit trail (`server/src/modules/audit/audit.service.ts`)

- `logAudit({ actorType, actorName, action, entityType, entityId, summary, details })` inserts one row. Called after **every** agent decision and human action.
- `actor_type` is `agent` ("Triage Agent", "Dispatch Agent"), `human` (the user's name) or `system` (seed script).
- `summary` is one readable sentence starting with a verb; `details` (JSONB) keeps the raw data, e.g. all candidates with their scores.
- **Incident History** (`getIncidentHistory`) shows entries about the incident **plus** its reports **plus** its dispatch decisions, oldest first.

- Action names used: `report.classified`, `report.merged`, `incident.created`, `incident.reclassified`, `incident.classification_confirmed`, `dispatch.recommended`, `dispatch.no_candidates`, `dispatch.accepted`, `dispatch.overridden`, `incident.on_scene`, `incident.resolved`, `system.seeded`.

Why the audit log matters: accountability (who decided what, when, why), trust in the AI (you can see how often humans override it), and it is the data for the end-term analytics (acceptance rate, response times).

10. Live updates with Socket.IO

REST loads the data once; after that the server **pushes** every change over a WebSocket, so all open screens update within about a second with no refresh and no polling. Server: `server/src/realtime/socket.ts`. Client: `client/src/hooks/SocketContext.tsx` and `useLiveData.ts`.

What is a WebSocket?

A normal HTTP request is one question, one answer, then the line closes, so the server cannot start a conversation. A **WebSocket** keeps one connection open in both directions, so the server can send a message whenever something changes. **Socket.IO** is a library on top of it that adds **events with names**, **rooms**, and **automatic reconnection**.

Server side

- Socket.IO is attached to the **same HTTP server and port (4000)** as the REST API.
- **Login check on connect:** the client sends its JWT in `socket.handshake.auth.token`; `io.use(...)` verifies it with the same `verifyToken()` used for REST. No valid token → connection refused.
- **Rooms:** every user joins `user:<id>` (private channel). Commanders and admins also join **`control-room`**.
- `broadcast(event, data)` → sends to the `control-room`. `sendToUser(id, event, data)` → sends to one user.

Event	Sent to	When
`incident:updated`	control room	incident created, merged, reviewed, recommended, dispatched, arrived, resolved
`resource:updated`	control room	unit added, edited, retired/restored, dispatched, arrived, freed

Event	Sent to	When
`decision:updated`	control room	recommendation created, accepted or overridden
`feed:status`	control room	social feed started, stopped, rewound, or posted the next item
`task:assigned`	the responder(s) linked to the unit	a commander dispatched that unit

The server always sends the **whole updated object** (not just "something changed"), read fresh from the database, so the client can simply replace its copy.

Client side

1. **`SocketProvider`** opens **one** connection after login (`io({ auth: { token }, transports: ['websocket'] })`) and shares it with every screen through React Context. It tracks `connected` (shown as the live indicator in the top bar). On logout it disconnects.
2. **`useLiveData()`** loads incidents, resources, feed status and AI/rules mode with 4 REST calls in parallel (`Promise.all`), then listens:
   - `incident:updated` → **upsert** (replace if it exists, add if new) and re-sort by urgency; if `resolved` , remove it from the queue.
   - `resource:updated` → upsert; if retired, remove it.
   - `feed:status` → replace the feed status.
3. **After a reconnect** (Wi-Fi drop, server restart) it **reloads everything over REST**, because events sent while offline are lost.
4. Listeners are removed with `socket.off` when the component unmounts, to avoid duplicates and memory leaks.

Same sort on both sides: urgency score high → low, then oldest first. Ties go to the older incident so it is not starved by newer ones.

The mock social media feed (`server/src/modules/feeds/socialFeed.ts`)

- 26 made-up posts (`server/src/data/socialPosts.ts`) at real Chennai localities. Some are tiny offsets (`dLat` , `dLng` , about 50–150 m) from each other on purpose, to test merging. One is noise ("Chennai rains are something else today!") to test the `other` type.
- `setInterval` posts the next one every `SOCIAL_FEED_INTERVAL_SECONDS` (5 s) through the **same** `ingestReport()` pipeline as the form, with `source: 'social_media'` .

- Endpoints: `POST /api/feeds/social/start`, `/stop`, `/rewind` (commander, admin); each is audited. Progress ("posted 7 of 26") is pushed as `feed:status`.
- Its progress is kept in memory, so restarting the server resets it.

11. Frontend: the React app

The frontend is a **single-page app** in React 19 + TypeScript: 5 pages, about 12 components, one CSS file per component, no UI library and no animations. Code: `client/src/`.

Pages and who can open them (`App.tsx`)

URL	Page	Roles	What it shows
`/login`	`LoginPage`	everyone	email + password form, demo accounts
`/`	`DashboardPage`	commander, admin	the control room: queue, map, incident details
`/reports/new`	`NewReportPage`	commander, admin	"Log a report" form with address search and map pin
`/resources`	`ResourcesPage`	admin	table of all units; add, edit, retire, restore
`/responder`	`ResponderPage`	responder	"Your unit", its status and whether it has a task (basic version)

After login each role lands on its home page (`homePathFor`): commander → `/`, admin → `/resources`, responder → `/responder`. Any unknown URL redirects to `/`.

Wrapping order: `BrowserRouter` → `AuthProvider` (who is logged in) → `SocketProvider` (live connection) → `Routes` → `Layout` (top bar with nav links and live indicator) → page inside `RequireRole`.

The dashboard (`pages/DashboardPage.tsx`): three columns

- **Left: queue.** `FeedControl` (start/pause the social feed), a note if triage is in keyword mode, and `IncidentList`: cards sorted by priority with type, severity, score, report count, status and a "Needs checking" badge.
- **Middle: map.** `MapView` with Leaflet: incidents are **circles coloured by severity** (bigger with more reports); units are **small squares with a letter** (A ambulance, T team, S supply truck, H shelter) **coloured by status**; a dashed blue **route line** for the suggested

or dispatched unit; a legend; a "Show shelters" checkbox; the map pans to the selected incident.

- **Right: details.** `IncidentPanel` for the selected incident (or `ResourcePanel` with unit counts when nothing is selected). Inside it:
  - `ClassificationReview`: "Is this classification correct?" → **"Yes, looks right"** or **"No, change it"** (type + severity dropdowns, Save).
  - `DispatchSection`: **"Find the best unit"** → recommendation card with the rationale and **"Send AMB-04"** / **"Choose another unit"** (pick a unit, "Reason (optional)", **"Send selected unit"**), "Other units considered", **"Ask for a fresh suggestion"**. Then **"Mark as arrived"**, **"Mark as resolved"**, and **"Close this incident"** for false alarms (with a confirm dialog).
  - The incident's reports and its **History** (audit log).
- `IncidentPanel` is rendered with `key={selectedId}`, so switching incidents gives a fresh component with clean local state.

Log a report (`pages/NewReportPage.tsx`)

Type what happened (min 10 characters), optional caller name, then set the location by **searching an address** (server proxy to Nominatim, results biased to a Chennai box) or **clicking the map** (`LocationPicker`, reverse-geocoded into an address). On submit it shows the result: new incident or merged into an existing one, the type and severity found, and whether it needs checking.

How the client talks to the server

- **`api/client.ts`**: a small `fetch` wrapper. Adds `Authorization: Bearer <token>`; turns error responses into an `ApiError` carrying the server's message (shown to the user as is); on **401** it logs the user out; if the server is unreachable: "Cannot reach the server. Check your connection and try again."
- **`auth/AuthContext.tsx`**: keeps `user` and `token`; the token is saved in **`localStorage`** (`drrc_token`); on page reload it calls `GET /api/auth/me` to restore the session.
- **`auth/RequireRole.tsx`**: not logged in → redirect to `/login`; wrong role → redirect to your own home page. This only **hides** pages; the real protection is on the server.
- **Live data:** `useLiveData()` (section 10). Screens never poll.
- **`lib/labels.ts`**: every database code is shown in plain words, e.g. `open` → "Waiting for a unit", `en_route` → "On the way", `personnel` → "Response team", `manual` → "Phone / walk-in". No raw codes on screen.
- **Permissions in the UI:** `canDispatch = user.role === 'commander'`. An admin sees the same dashboard but **without action buttons** (read-only).

React ideas to be able to explain

- **Component:** a function that returns UI (JSX). **Props:** data passed in from the parent. **State (`useState`):** data that changes and re-renders the component.
- **`useEffect`:** run code after render (load data, subscribe to socket events) and clean up on unmount.
- **Context (`createContext` / `useContext`):** share data (logged-in user, socket) with every component without passing props down each level.
- **Custom hook (`useLiveData`, `useAuth`, `useSocket`):** reusable logic that uses other hooks.
- **Vite:** dev server with instant reload; `npm run build` makes the production bundle (about 500 kB; code-splitting is a later improvement).
- **Why a design without animations or gradients?** Control-room staff are under stress; calm, plain screens with clear words are faster to read and look like a real tool.

12. Security and roles

Permissions are enforced **on the server** for every request; the frontend only hides buttons and pages for convenience. If someone calls the API directly with the wrong role, they get **403**.

Who can do what

Action	Commander	Admin	Responder
See the dashboard, queue, map, history	yes	yes (read-only)	no
Log a report, start/stop the social feed	yes	yes	no
Confirm/correct a classification	yes	no	no
Find a unit, send, override, arrived, resolved, close	yes	no	no
Add, edit, retire, restore resources	no	yes	no
See own unit only	(n/a)	(n/a)	yes

Why can't the admin dispatch? Separation of duties: the admin manages the inventory, the commander makes operational decisions. It also keeps the audit trail clear about who decided.

Example of a data-level rule: `GET /api/resources/:id` lets a responder see **only their own unit**; asking for another gives 403 "You can only view your own unit."

Security measures (list these if asked "how is it secure?")

1. **Passwords hashed with bcrypt** (cost 10, salted); never stored or logged in plain text.
2. **JWT** on every API call and on the WebSocket handshake; signed with a secret of at least 16 characters; **expires after 12 hours**.
3. **Role checks on the server** with `requireRole(...)` middleware.
4. **Input validation with zod** on every body, URL parameter and query string, plus **CHECK constraints** in the database as a second line of defence.
5. **Parameterised SQL** (`$1, $2`) everywhere, which prevents **SQL injection**.
6. **helmet** security headers; **CORS** limited to the frontend's address.
7. **Same error for wrong email and wrong password**, so attackers cannot find out which emails exist.
8. **Safe errors:** unexpected errors return a generic 500 message; stack traces are only in the server log.
9. **Transactions + row locks** stop double-booking and inconsistent states.
10. **Secrets in** `.env` , not in the code; `.env.example` shows the keys without values. The Gemini key and Nominatim calls stay **on the server**, never in the browser.
11. **Request size limit** of 1 MB on JSON bodies.

Honest weak spots (say them before the examiner does)

- The token is stored in `localStorage` , which a cross-site scripting (XSS) bug could read. An **httpOnly cookie** would be safer; React escapes text by default, which lowers the risk.
- **No rate limiting** on login yet (brute-force protection), and no refresh tokens or token revocation on logout.
- Development runs on **HTTP**; production would need HTTPS.
- Demo passwords are public in the docs; they are for the demo only.

13. What is done, what comes next, known limits

The mid-term half is complete: **report intake, triage, dedup, queue, dispatch with routing, human approval, lifecycle, audit, live map and roles**. The full list is 50 tasks in `TODO.md` , split into MID and END.

Done for mid-term (and tested)

- PostgreSQL + PostGIS schema with GIST indexes, migration runner, Chennai seed data.
- JWT login, 3 roles, server and client role guards.
- Manual report form with address search and map pin; mock social feed (26 posts, start/pause/rewind).
- Triage Agent (LangGraph, Gemini structured output, keyword fallback, confidence check) and urgency score with a reason.
- Deduplication with `ST_DWithin` + KNN, serialised queue.
- Commander classification review.
- Live incident queue sorted by priority.
- Resource inventory with add/edit/retire/restore (admin).
- Dispatch Agent with OSRM road ETAs, ranking and plain-language rationale; accept/override in one transaction.
- Lifecycle: arrived, resolved, close without a unit.
- Audit trail and incident History; live map with route line; resource status panel.
- Admin read-only dashboard; basic responder page.
- Docs: README with a demo script, User Guide, Architecture, API, Learning Guide, Improvement Scope.
- **Testing done:** TypeScript compiles with no errors on both sides, API tests of every flow including permission and conflict cases, and browser walkthroughs (desktop, phone width, two sessions at once) with zero console errors. There are **no automated test files yet**; they are an end-term task.

Planned for end-term

Feature	What it will do
Supervisor Agent	One LangGraph graph that runs Triage → Dispatch → Routing, detects conflicts (two incidents want the same unit) and applies an escalation policy
Responder mobile view	Assigned incident, route on a map, "arrived"/"resolved" buttons from the phone (server already sends `task:assigned`)
Vehicle simulation	Units move along the OSRM road path, positions pushed over sockets (`getRoute()` already returns the path)

Feature	What it will do
Notifications	Alert the commander for severity 4–5; alert the responder on assignment
Analytics	Response times (`created_at` → `dispatched_at` → `resolved_at`) and AI acceptance rate (`accepted` vs `overridden`); the data is already stored
Weather feed	Open-Meteo alerts as a third report source (`weather_api` is already an allowed source)
Smaller items	Image upload (`reports.image_url` exists), urgency rising with waiting time, re-prioritise, merge/split incidents, user management, multi-stop and re-routing, decision log page
Quality	Automated tests, one-click flood scenario, production build and deployment

Known limitations (be honest and give the fix)

Limit	Why it matters	Fix
AI path not yet run live (no Gemini key)	Demo uses keyword rules; everything is marked "Needs checking"	Add `GEMINI_API_KEY` , run `npm run ai:check`
In-memory state (dedup queue, feed progress, geocode cache)	Works for one server only; lost on restart	Database advisory locks, Redis
Needs internet (OSRM, Nominatim, map tiles, Gemini)	Disasters often cut the internet	Self-host OSRM and tiles; rules + estimate fallbacks already exist
One unit per incident	A big fire may need several teams	Multi-unit dispatch (schema already allows it: units point to the incident)
Units jump to the incident on arrival	No live tracking	Vehicle simulation (end-term), real GPS later
Simple urgency weights	Chosen by us, not learned	Tune with analytics data
Public OSRM server	Rate limits, not for production	Self-host (just change `OSRM_URL`)

Limit	Why it matters	Fix
Client bundle about 500 kB	Slower first load	Code-splitting

14. Demo script (about 8 minutes)

Run the demo in this order; each step shows one feature and has one sentence to say.

Before the viva (do this first)

- [ ] **Fix the database connection on this laptop.** PostgreSQL 18 is installed at `D:\Applications\PostgreSQL`, but the password in `server/.env` is wrong and **PostGIS is not installed**. Put the real password in `DATABASE_URL`, install PostGIS through Stack Builder, and create the `drrc` database.
- [ ] `npm run db:reset` (clean demo data), then `npm run dev`, open http://localhost:5173.
- [ ] Check the internet works (map tiles, OSRM, Nominatim need it).
- [ ] Optional: add `GEMINI_API_KEY` and run `npm run ai:check`. Without it, the app runs in keyword mode; say so honestly.
- [ ] Keep a second browser window ready for the admin login.

The demo

#	Do	Say
1	Log in as `commander@drrc.local` / `Commander@123`	"Each role gets a different screen; the server checks the role on every request."
2	Show the map and unit panel	"These are 33 real-location units across Chennai, coloured by status."
3	Click start on the social media feed	"Every 5 seconds a post comes in and goes through the Triage Agent."
4	Point at the queue and the Velachery/Saidapet incidents growing to "2 reports", "3 reports"	"Reports of the same type within 500 m and 60 minutes merge, using PostGIS. More reports raise the priority."
5	Point at "Needs checking"	"Low-confidence results are never silently accepted; a human checks them."

#	Do	Say
6	Open the top incident; read the priority and classification reasons; click "Yes, looks right" or correct it	"Every score comes with a reason anyone can verify."
7	Click "Find the best unit"	"The Dispatch Agent finds the nearest suitable free units with a KNN query, then gets real road times from OSRM."
8	Show the route line, ETA, rationale, "Other units considered"	"It explains why, including when a unit looks closer but is slower by road."
9	Click "Send …"	"Nothing moves until the commander clicks. It runs in one transaction with row locks."
10	On another incident: "Choose another unit", pick one, type a reason, send	"The human can override the agent; the reason is logged."
11	"Mark as arrived", then "Mark as resolved"	"The unit becomes available again automatically."
12	Scroll to History	"Every agent decision and human action is in the audit log with time and actor."
13	"Log a report": type a phone report, search "Velachery", save	"Manual reports use the same pipeline; here it merged into the existing incident."
14	Second window: log in as `admin@drrc.local` / `Admin@123`, edit a resource	"The commander's screen updated live through Socket.IO, no refresh."
15	Optional: `responder1@drrc.local` / `Responder@123` at phone width	"The responder sees only their unit; the full mobile view is end-term."

If something goes wrong

- **"Cannot connect to the database"** in the server log → `DATABASE_URL` is wrong or PostgreSQL is stopped.
- **ETA says "rough estimates"** → OSRM is unreachable; this is the designed fallback. Say: "This is our fallback: straight line × 1.4 at 25 km/h, clearly labelled."
- **Map tiles grey** → no internet; the data still works.

- **Address search fails** → "Address lookup is not available right now. Please pick the spot on the map." Click the map instead.
- **Everything says "Needs checking"** → keyword mode (no Gemini key). Explain it is by design.
- **Messy state** → stop, `npm run db:reset`, start again.

15. Viva question bank

Short answers, grouped by topic. If you don't know something, say what the code does and where it lives; examiners respect "it's in `dispatch.service.ts`, the `decide()` function" more than a guess.

Team split (from the SRS, §8): A = ingestion and triage (reports, feed, triage, dedup, urgency, queue). B = agents and dispatch (dispatch, routing, accept/override, audit, resources). C = real-time and frontend (sockets, dashboard, map, login, database setup). Everyone must explain the two main flows: a report arriving, and a unit being dispatched.

Project and design

Q: What problem does it solve? In a disaster, reports pour in from many channels and many describe the same event. We classify, merge, rank and suggest units automatically, and keep a human in charge of every dispatch.

Q: What makes something an "agent" here? It takes an input, decides something on its own using a model or rules, and returns a decision **with a reason**. Triage decides type and severity; Dispatch decides the best unit; Routing decides travel times. None of them act on the world; they recommend.

Q: Why not let the AI dispatch automatically? Wrong dispatches cost lives and AI can be wrong. The SRS requires human override for critical actions. Agents recommend with a reason; the commander accepts or overrides; both are logged.

Q: What is "human in the loop"? A person must approve before the system acts. Here: no unit changes status until a commander clicks Send.

Q: What does "explainable" mean in your project? Every automatic decision comes with a sentence anyone can check: the triage reason, the urgency reason ("Severity 4 gives 60 pts…") and the dispatch rationale ("9 min by road, 2 min faster than…").

Q: Why Chennai? Real flood history (monsoon), many localities, rivers that make straight-line distance misleading. Coordinates come from OpenStreetMap; units, people and posts are fictional.

Q: Is the data real? Locations are real (OpenStreetMap). Units, users and social media posts are made up.

Triage and AI

Q: What is an LLM? A large language model: a neural network trained on huge amounts of text that predicts text. We use Google Gemini to read a report and classify it.

Q: What is structured output? We give the model a schema (zod) and it must answer in exactly that JSON shape: type, severity, confidence, title, reason. We validate it again. No fragile text parsing.

Q: Why temperature 0? We want the same answer for the same report, not creativity.

Q: What if Gemini is down or there's no key? The LangGraph conditional edge routes to the keyword classifier. Its confidence is at most 0.5, so the result is always marked "Needs checking". The system never stops working.

Q: What is a prompt? What's in yours? The instructions sent to the model. Ours says it is a Chennai control-room triage officer, lists the 8 types and the severity scale, says to use only the text, pick the most life-threatening problem, accept Tamil words in English letters, and give low confidence when unsure.

Q: What are LangGraph nodes, edges and state? Nodes are functions (classify with AI, classify with rules, check confidence). Edges decide what runs next; a conditional edge decides at run time. State is the shared object all nodes read and update.

Q: Have you tested the AI? Honest answer: the code is complete and type-checked and `npm run ai:check` exists to test a key, but we haven't added our Gemini key yet, so the demo runs in keyword mode. (Change this answer if you add the key before the viva.)

Q: Can the AI hallucinate a wrong type? It can misjudge, but it cannot invent a type outside our 8, because the schema only allows those values. Low confidence goes to human review, and the commander can correct any classification.

Q: How is urgency calculated? Severity × 15 + type points (3 to 15) + 2 per extra report (max 10), capped at 100. Example: severity-4 flood with 3 reports = 60 + 10 + 4 = 74.

Q: Why is the score not learned by machine learning? No historical data yet, and a hand-made formula is explainable. The weights can be tuned later using the analytics.

Deduplication and PostGIS

Q: How do you detect duplicates? Same type, within 500 m (`ST_DWithin`), last report within 60 minutes, not resolved, nearest first (`<->`). Found → merge and raise report count; else create. Both limits are in `.env` .

Q: Why is `other` never merged? Two vague reports near each other may be about different things.

Q: What is a race condition and where could you have one? When the result depends on timing. Two reports of the same event arriving together could both miss each other and create two incidents. We run the dedup step one at a time through a promise queue.

Q: What is PostGIS? An extension that adds geographic types, functions (distance, within, nearest) and spatial indexes to PostgreSQL.

Q: Geography vs geometry? Geography measures on the curved earth in metres; geometry is flat maths in the coordinate's units (degrees). We store geography because we need metres.

Q: What is SRID 4326? The ID of the WGS 84 coordinate system used by GPS (latitude/longitude).

Q: What is a GIST index? A spatial index that groups nearby points into boxes, so "within 500 m" and "nearest first" check only nearby boxes instead of every row.

Q: What does `<->` do? The KNN distance operator. In `ORDER BY` it returns rows nearest first and can use the GIST index.

Q: Why `ST_MakePoint(lng, lat)` in that order? It takes x then y; longitude is x. Swapping them puts Chennai in the ocean.

Dispatch and routing

Q: How does the Dispatch Agent choose? Requirements by type → 8 nearest free units of the right type (KNN) → prefer those with the needed equipment, keep 4 → OSRM road ETAs → score = ETA minutes + 10 per missing capability + 3 × load ratio → lowest score wins → plain-language reason.

Q: Why road ETA and not distance? Nearest is not always fastest: rivers, bridges and one-way roads. The rationale even says when a unit "looks nearer on the map but takes longer by road".

Q: What is OSRM? Open Source Routing Machine: a fast routing engine on OpenStreetMap roads. We use its **table** service (many-to-one travel times in one request) and **route** service (the path for the map).

Q: What if OSRM is down? Fallback: straight-line distance (haversine) × 1.4 at 25 km/h, labelled as an estimate in the rationale.

Q: What is the haversine formula? Distance between two GPS points on a sphere (earth radius 6,371 km). Used only in the fallback; normal distances are computed by PostGIS.

Q: What if no unit is free? The agent returns nothing, the audit log records it, and the commander sees "No suitable unit is available right now. Try again when a unit is free."

Q: What is stored for each recommendation? A `dispatch_decisions` row: the recommended unit, all candidates as JSONB with ETAs and scores, the rationale, and later the human action, unit sent, reason, who and when.

Database and backend

Q: What is a transaction? Several statements that succeed or fail together (`BEGIN` … `COMMIT`, or `ROLLBACK` on error). ACID: atomic, consistent, isolated, durable.

Q: Two commanders click Send on the same unit at once? `decide()` locks the decision, incident and unit rows with `SELECT … FOR UPDATE`. The second waits, then sees the unit is no longer available (or the decision already answered) and gets a 409. Never double-booked.

Q: What is a connection pool? A set of open database connections reused by all requests, because opening a new one each time is slow.

Q: How do you prevent SQL injection? Parameterised queries: `$1, $2` with values passed separately, never pasted into the SQL string.

Q: Why no ORM? Plain SQL is transparent and PostGIS functions are easier to write directly; the SRS also specifies PostgreSQL + PostGIS.

Q: What is middleware? A function that runs before the route handler; it can stop the request or add data. Ours: helmet, cors, JSON parser, logger, `requireAuth`, `requireRole`, error handler.

Q: REST vs WebSocket: why both? REST for request/response actions and loading; WebSocket for the server to push changes. Each does what it is best at.

Q: What HTTP status codes do you use? 200/201 success, 400 bad input, 401 not logged in, 403 wrong role, 404 not found, 409 conflict (already done by someone else), 500 unexpected, 502 address lookup down.

Q: What is a migration? A versioned SQL file that changes the schema. Our runner applies new files in order inside a transaction and records them in `schema_migrations`.

Auth and security

Q: How does login work? bcrypt compares the password with the stored hash → server signs a JWT (id, name, role, unit) valid 12 h → browser sends it as `Authorization: Bearer` on every request and on the socket handshake.

Q: What is inside a JWT? Is it encrypted? Header, payload, signature. It is **signed, not encrypted**: anyone can read the payload, but nobody can change it without the secret. So no passwords in it.

Q: Why bcrypt and not SHA-256? bcrypt is deliberately slow and salted, so brute force and rainbow tables are impractical.

Q: How is it secured? bcrypt, JWT, server-side role checks, zod validation + CHECK constraints, parameterised SQL, helmet, CORS, same error for wrong email/password, transactions with locks. Weak spots: token in localStorage, no rate limiting yet, HTTP in development.

Frontend and real-time

Q: How do updates reach other screens? After each change the server emits a Socket.IO event with the full updated object to the `control-room` room; `useLiveData` upserts it and re-sorts. After a reconnect the client reloads everything over REST.

Q: What are Socket.IO rooms? Named groups of connections. `control-room` = commanders + admins; `user:<id>` = one user, used for `task:assigned`.

Q: Why React? Component-based, huge community, and the UI is built from state, so live updates just change the state and React redraws.

Q: What is React Context? A custom hook? Context shares values (user, socket) with all components without passing props. A custom hook (`useLiveData`) packages reusable stateful logic.

Q: Why Leaflet and OpenStreetMap? Free, open, no API key, light, and good with GPS coordinates.

Q: Why does the UI avoid animations and gradients? Control-room users are under stress; calm screens, plain words and colour only where it means something are faster to read.

Scale and future

Q: How would it scale to a real city? Self-hosted OSRM, a job queue for triage, a Redis adapter for Socket.IO across several servers, database locks instead of the in-memory queue, pagination and read replicas.

Q: What would you improve first? Run the AI live, add the Supervisor Agent for unit conflicts, the responder mobile view, and automated tests.

Q: What is the Supervisor Agent? End-term: one LangGraph graph that runs Triage → Dispatch → Routing, detects when two incidents want the same unit, and escalates by policy. Still recommends only.

Q: What did you learn? Geospatial queries, designing explainable AI with a fallback, transactions and locking for concurrency, and real-time push.